

How to Merge Your Multimodal Models Over Time? 🕒

Sebastian Dziadzio^{*1} Vishaal Udandarao^{*1,2} Karsten Roth^{*1,3} Ameya Prabhu^{o1}
 Zeynep Akata^{3†} Samuel Albanie[†] Matthias Bethge^{1†}

¹Tübingen AI Center, University of Tübingen ²University of Cambridge ³Munich Center for ML, Technical University of Munich

Abstract

Model merging combines multiple “expert” models—finetuned from a base foundation model on diverse tasks and domains—into a single, more capable model. However, most existing model merging approaches assume that all experts are available simultaneously. In reality, new tasks and domains emerge progressively over time, requiring strategies to integrate the knowledge of expert models as they become available: a process we call *temporal model merging*. The temporal dimension introduces unique challenges not addressed in prior work, raising new questions such as: when training for a new task, should the expert model start from the merged past experts or from the original base model? Should we merge all models at each time step? Which merging techniques are best suited for temporal merging? Should different strategies be used to initialize the training and deploy the model? To answer these questions, we propose a unified framework called **TIME**—*Temporal Integration of Model Expertise*—which defines temporal model merging across three axes: (1) Initialization Phase, (2) Deployment Phase, and (3) Merging Technique. Using TIME, we study temporal model merging across model sizes, compute budgets, and learning horizons on the FoMo-in-Flux benchmark. Our comprehensive suite of experiments across TIME allows us to uncover key insights for temporal model merging, offering a better understanding of current challenges and best practices for effective temporal model merging. Our code is available [here](#).

1. Introduction

Foundation models consolidate a wide range of capabilities and knowledge into a single, large model [5]. Consequently, model merging [24, 34, 63, 87] has emerged as a key technique for unifying multiple task-specific fine-tuned models

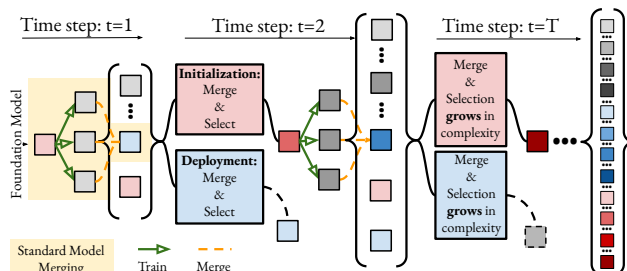


Figure 1. **Temporal Model Merging** generalizes standard model merging (yellow), which merges multiple trained experts just once, in a single step. Our systematic study of this realistic multi-step regime reveals that initialization and deployment strategies dominate the importance of the single-step weight merging strategy.

derived from a shared base into a single model, effectively preserving the strengths of each specialized expert.

Current model merging approaches typically assume a fixed base model that is fine-tuned independently on k diverse tasks and domains to produce a set of *independent* experts [17, 24, 34, 64, 89], which are then merged simultaneously. Research in this field has therefore focused primarily on improving merging techniques for larger or structurally differing k -sets, exploring the impact of the diversity and scale of finetuning domains, tasks and experts [27, 69, 86, 89].

However, the world is constantly evolving, leading to continuous shifts over data distributions, domains, and tasks, with new concepts emerging [45, 68] that may have been insufficiently covered during large-scale pretraining [20, 22, 32, 45, 57, 65, 66, 81–83, 98]. This dynamic nature of real-world applications motivates a hitherto missing systematic exploration into *temporal model merging* (see Fig. 1) to better understand model merging along an additional, overlooked [13, 100] axis: *time*. Specifically, in this work, we ask: (1) Is temporal model merging significantly influenced by the choice of initialization for expert training? (2) When evaluated over time, which model merging techniques emerge as most suitable? (3) Should merging strategies differ between initialization and model deployment? To answer these questions, we propose a unified framework

^{*}equal contribution, random order [†]equal supervision, random order,
^o core contributor.
 Correspondence to: vishaal.udandarao@bethgelab.org or
 sebastian.dziadzio@bethgelab.org

for studying temporal model merging—TIME (Temporal Integration of Model Expertise)—structured around three key axes spanning the design space of temporal merging solutions (as shown in Fig. 2):

1. Initialization Phase. As expert models are trained and created continuously over time, initialization becomes a crucial design element.

2. Deployment Phase. After training an expert on each task, the next step is to deploy a suitable final model. For temporal model merging, this process has to account for a varying number of past expert models and deployed variants, striking a balance between retaining past knowledge and incorporating new task-specific knowledge.

3. Model Merging Techniques. Merging literature has explored a range of approaches for simultaneous merging, from simple weight-averaging and interpolation [24, 63, 68, 76] to more complex methods involving weight and candidate selection and re-weighting [11, 42, 43, 89, 95]. To apply these methods temporally, it is crucial to understand how they perform with varying numbers of merge candidates (k_t) and shifting distributions.

Using our TIME framework, we position existing model merging approaches along each key axis and conduct a systematic study of model merging over time. For this, we leverage the multimodal FoMo-in-Flux [68] benchmark which includes 63 datasets with well-documented sequential properties, enabling a thorough investigation of temporal model merging under practical compute constraints, as proposed in Roth et al. [68]. Our experiments systematically explore different merging techniques, initialization, and deployment strategies, providing several key insights:

Key Insights for Temporal Model Merging

[A] Accounting for time is crucial. Standard “*offline*” model merging techniques do not generalize well to the temporal merging setting (Sec. 4.2).

[B] Complex merging techniques matter little. Choosing sophisticated merging techniques beyond simple weighted averaging provides at best marginal benefits for temporal model merging, especially for long task sequences (Sec. 4.5).

[C] Initialization and deployment are critical. Choosing how to select and combine available weights before and after each task t is most important for temporal model merging (Sec. 4.4).

[D] Temporal model merging scales well. Larger models or compute budgets allows greater benefits from temporal merging. Scaling enables temporal model merging to even outperform the multitask model, trained on all tasks at once (Sec. 4.6).

2. Related Works

Model Merging. We provide a short overview of the model merging literature, detailed in these excellent surveys [90, 92]. While both model aggregation through distillation [10, 67] and averaging checkpoints during training [29, 35, 69] have shown success, the requirement of additional compute limits practicability of these methods [55]. Instead, recent work [18, 21, 24, 25, 41, 47, 49, 50, 52, 59, 69, 71, 72, 75, 77, 85–88, 90, 93, 96, 99] has shown the effectiveness of training-free weight averaging and interpolation of fine-tuned expert models to produce an improved base model, benefiting from (linear) mode connectivity in models fine-tuned from a single pre-trained checkpoint [1, 14, 26, 48, 61]. These insights have been extended into weight-averaged reward models [62], policy models [60] with spherical interpolation, and KL-constrained RLHF [19, 36, 39, 46]. Works such as Fisher-Merge [43], TIES [89], RegMean [28], MATS [78], DELLA [12], DARE [95], Breadcrumbs [11], evolutionary merging [2] and MagMax [42] have explored merging strategies beyond simple interpolation to determine which weights should be merged across expert models. These methods have different benefits for in- and out-of-distribution generalization across domains [79], though recently they have been shown to perform similarly at scale [91]. Additionally, some works have explored the initialization dimension for effectively merging models [9, 13, 42, 100]. In this work, we propose a unifying framework for temporal merging and conduct the most comprehensive study of this topic to date.

Continual Pretraining extends beyond standard Continual Learning [54, 68], focusing on large-scale model updates starting from pretrained foundation models [16, 20, 23, 56, 68] and addressing more complex and substantial update tasks [6, 7, 16, 37, 38, 68]. There has been limited exploration into using model merging for continual pretraining [3, 42, 68, 76], as most prior works focus on training strategies including regularization objectives and learning-rate schedules [16, 23, 33, 44, 51, 55, 68, 74, 80, 94]. We keep the training strategy fixed, and provide an in-depth exploration beyond simple merging techniques.

3. Design Space of Temporal Model Merging

Notation. Throughout this work, we use t to refer to a given task at time t . Full model parameterization is denoted by θ , with the following key instantiations: θ_t represents model weights at task t , while θ_t^I , θ_t^S , and θ_t^O denote weights used for *initialization*, *saved* weight checkpoints at task t (i.e. the trained expert models), and the *output* deployed model weights, respectively. Note that while standard model merging considers model weights as elements of a fixed set $\{\theta_k\}_{k=1}^K$, temporal model merging organizes them along the time axis θ_t .

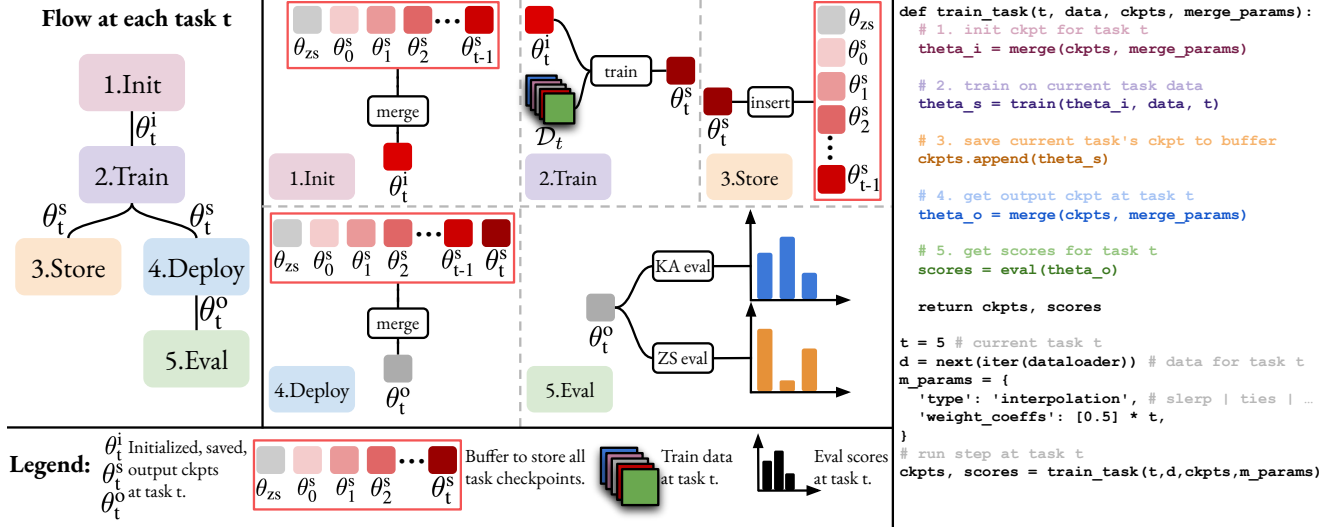


Figure 2. **Design Space of Temporal Model Merging through TIME.** We showcase our framework for the per-task pipeline of temporal model merging over multiple tasks: At each task t , we first initialize the current checkpoint to start training from, θ_t^i , by using one or more previously stored checkpoints from previous tasks, either directly or by merging them. We train θ_t^i on current task data $\mathcal{D}_t$ to yield the current task checkpoint θ_t^s , which is inserted into the checkpoint buffer. Finally, to produce the output model, θ_t^o , we either merge previously stored checkpoints from the buffer or use them directly. The entire framework is depicted in the pseudo-code on the right panel.

3.1. Temporal Model Merging through TIME

Standard model merging is typically performed offline, after all experts have been trained to convergence [15, 87, 89, 91]. In contrast, model merging in continual pre-training is generally done sequentially, using past checkpoints [68, 76]. Both approaches are specific instances of our more general temporal merging framework, TIME, which defines temporal merging along *three key axes*: initialization of each expert, merging for deployment at step t , and merging techniques f_{merge} applied over time:

Axis 1: Initialization

As expert models are created continuously over time, initialization becomes a crucial choice. Unlike model merging at a single point in time, the number of potential starting points grows exponentially over time as new experts are created. This raises the question: should starting points for each time step be derived from the base weights (as in traditional merging), from a merged combination of previous experts, or from most recent weights, as commonly done in continual pretraining [68, 76]? In this work, we study the following initialization protocols at time step t for TIME:

- init_{zs} , which consistently initializes with the base zero-shot model weights θ_0 at each timestep t .
- init_{FT} , which for step t always initializes with the latest available finetuned model weights θ_{t-1}^s .
- init_{EMA} , which computes an unrolled exponential moving average merge over all previously seen expert models $\{\theta_{t'}^s\}_{1, \dots, t-1}$ following the equation:

$$\theta_{t'}^{\text{EMA}} = f_{\text{merge}}(\theta_{t'-1}^{\text{EMA}}, \theta_{t'-1}^s, \mathcal{F}) \quad (1)$$

with merging hyperparameters $\mathcal{F}$. Consequently, the initialization weights are given as $\theta_t^i = \theta_t^{\text{EMA}}$.

Axis 2: Deployment

With each update iteration and expert training phase t , a decision must be made on the final model to deploy, determining which weights to present for downstream use. In continual pretraining, the trained model θ_t^s is deployed directly. In contrast, standard model merging applies a merging technique f_{merge} to a fixed set of k expert models. Temporal model merging, however, must account for both previously deployed models and the growing number of expert models available over time. Unlike standard merging, where k remains constant, the number of experts to merge increases with each step. As a result, temporal merging introduces the idea of weighted combinations, balancing recent updates with retained past knowledge to achieve adaptability and stability—both critical for effective continual learning [31, 54, 55, 68, 97]. In this work, we study three strategies for model deployment:

- $\text{deploy}_{\text{FT}}$, which at step t deploys the latest finetuned expert model, i.e. $\theta_t^o = \theta_t^s$.
- $\text{deploy}_{\text{EMA}}$, which computes an unrolled exponential moving average merge over all expert models, i.e. $\theta_t^o = \theta_{t+1}^{\text{EMA}}$ following Eq. (1).
- $\text{deploy}_{\text{ALL}}$, which applies a merging technique f_{merge} over all previously computed expert models $\{\theta_{t'}^s\}_{1}^{t-1}$.

Axis 3: Merging Techniques

At each point in time, for both initialization and deployment, merging technique f_{merge} defines *how* to combine

Table 1. Comparison of model merging techniques.

Method	Sparsification	Consensus	Scaling
Weight averaging [63, 86]	$\times$	Linear Int.	Weight coeff.
SLERP [60]	$\times$	Spherical Int.	Weight coeff.
Task Arithmetic [25]	$\times$	Linear Int.	Scaling factor
MagMax [42]	$\times$	Max. Magnitude	Scaling factor
TIES [89]	Top-k	Sign Agreement	Scaling factor
DARE-TIES [95]	Random	Sign Agreement	Scaling factor
Breadcrumbs-TIES [11]	Top/Bottom-k	Sign Agreement	Scaling factor
Model Stock [27]	$\times$	Geometric	Adaptive ratio
LiNeS [84]	$\times$	$\times$	Layer weights

the available expert models and checkpoints. In this work, we study nine different variants in total, shown in Tab. 1. Denoting the number of models to merge at timestep t as M_t (with $t = 0$ and $M_t = M$ for standard model merging), we can define these methods as follows:

Weight Averaging [24, 63, 68, 76, 87] simply employs a uniformly weighted, element-wise average over all models $\theta_{t,i}$, resulting in a merge function $f_{\text{merge}}^{\text{WA}}$:

$$\theta_t = \frac{1}{M_t} \sum_i \theta_{t,i}. \quad (2)$$

SLERP [60, 73] assumes weights to live on a hypersphere, and consequently conducts interpolation along a curved path connecting weight entries. In particular, for two models $\theta_{t,1}$ and $\theta_{t,2}$ deriving from some base weight θ_{t-1} and the corresponding task vectors [25] $\delta_{t,i} = \theta_{t,i} - \theta_{t-1}$, SLERP with interpolation weight λ is defined as

$$\theta_t = \theta_{t-1} + \frac{\sin(1-\lambda)\Omega_{1,2}}{\sin \Omega_{1,2}} \cdot \delta_{t,1} + \frac{\sin \lambda\Omega_{1,2}}{\sin \Omega_{1,2}} \cdot \delta_{t,2} \quad (3)$$

with $\Omega_{1,2}$ being the angle between task vectors $\delta_{t,1}$ and $\delta_{t,2}$. We denote the corresponding merge function $f_{\text{merge}}^{\text{SLERP}}$.

Task Arithmetic [25] defines the merge as a function over task vectors $\delta_{t,i} = \theta_{t,i} - \theta_{t-1}$ for each weight $\theta_{t,i}$ fine-tuned from θ_{t-1} . This introduces a simple merge formalism $f_{\text{merge}}^{\text{TA}}$ for weighted parameter averaging with a scale λ :

$$\theta_t = \theta_{t-1} + \lambda \frac{1}{M_t} \sum_i \delta_{t,i} \quad (4)$$

TIES [89] builds on the task arithmetic formalism through controlled pruning of task vector entries with low magnitude. Moreover, the sign for each final merged parameter is set based on the sign of the highest total magnitude across the merge candidates. The final update follows basic task arithmetic, only for entries with matching signs. We refer to the respective merge function as $f_{\text{merge}}^{\text{TIES}}$.

DARE [95] is a similar extension of task arithmetic, but instead of targetted pruning, it randomly zeroes out task vector entries using a random mask $Z_i \sim \text{Bernoulli}(p)$ and

masking probability p . Final task vector values for $f_{\text{merge}}^{\text{DARE}}$ are then rescaled based on p :

$$\delta_{t,i}^{\text{DARE}} = \frac{(1 - Z_i)\delta_{t,i}}{1 - p}. \quad (5)$$

Model Stock [27] provides a geometric extension of simple weight averaging as done in Model Soup [86] by incorporating base weights θ_{t-1} into the merging process. Given fine-tuned weights $\theta_{t,1}$ and $\theta_{t,2}$, the Model Stock merge $f_{\text{merge}}^{\text{Stock}}$ is defined as follows:

$$\theta_t = \frac{2 \cdot \cos \Omega_{1,2}}{1 + \Omega_{1,2}} \cdot (\theta_{t,2} - \theta_{t,1}) + \left(1 - \frac{2 \cdot \cos \Omega_{1,2}}{1 + \cos \Omega_{1,2}}\right), \quad (6)$$

utilizing angle $\Omega_{1,2}$ between task vectors $\delta_{t,1}$ and $\delta_{t,2}$.

Breadcrumbs [11] deploys another variation on task arithmetic for model merging. In particular, for a given task vector $\delta_{t,i}$, extreme left and right tails of the absolute magnitude distribution in $\delta_{t,i}$ are zeroed out with left and right thresholds β and γ . The modified task vectors $\delta_{t,i}^{\text{Bread}}$ are then applied on base weights θ_{t-1} following the task arithmetic setup, and giving $f_{\text{merge}}^{\text{Bread}}$.

MagMax [42] also uses task vectors—given multiple task vectors $\delta_{t,i}$ (with increments possible along both time t and count axis i), the final task vector δ_t is yielded through maximum magnitude entry selection; copying the largest magnitude entries across all $\{\delta_{t,i}\}$ into δ_t , giving $f_{\text{merge}}^{\text{Max}}$.

LiNeS [84], for Layer-increasing Network Scaling, scales weight updates based on their respective layer depth enabling early layers to remain close to original pretraining weights (cf. Neyshabur et al. [48]). Given task vectors $\delta_{t,i}$, now broken down across model layers $\delta_{t,i}^l$ with $l \in [1, \dots, L]$ and L the number of layers, LiNeS follows the base task arithmetic merging formalism, but updates task vectors as

$$\delta_{t,i}^{\text{LiNeS}} = \text{concat}(\lambda^{l=1}\delta_{t,i}^{l=1}, \dots, \lambda^{l=L}\delta_{t,i}^{l=L}) \quad (7)$$

with layer-scaled interpolation weights $\lambda^l = \alpha + \beta \frac{l-1}{L-1}$ and hyperparameters α, β , giving $f_{\text{merge}}^{\text{Lines}}$.

3.2. Complete Temporal Model Merging Pipeline

Incorporating all three axes of temporal model merging, we define a five-stage update pipeline for each task t (see Fig. 2), consisting of the following steps:

1. **Init.** Choose one of the aforementioned initialization protocols: `initZS`, `initFT`, or `initEMA`. This produces initialization weights θ_t^I at task t .
2. **Train.** Given θ_t^I , train on current task data $\mathcal{D}_t$ within a set compute budget to produce the expert model θ_t^S .
3. **Store.** Append θ_t^S to storage of expert model weights $\mathcal{S}$.
4. **Deploy.** Choose a deployment protocol: `deployFT`, `deployEMA`, or `deployALL`, and produce the output weights $\theta_t^O = \text{deploy}(\mathcal{S})$.

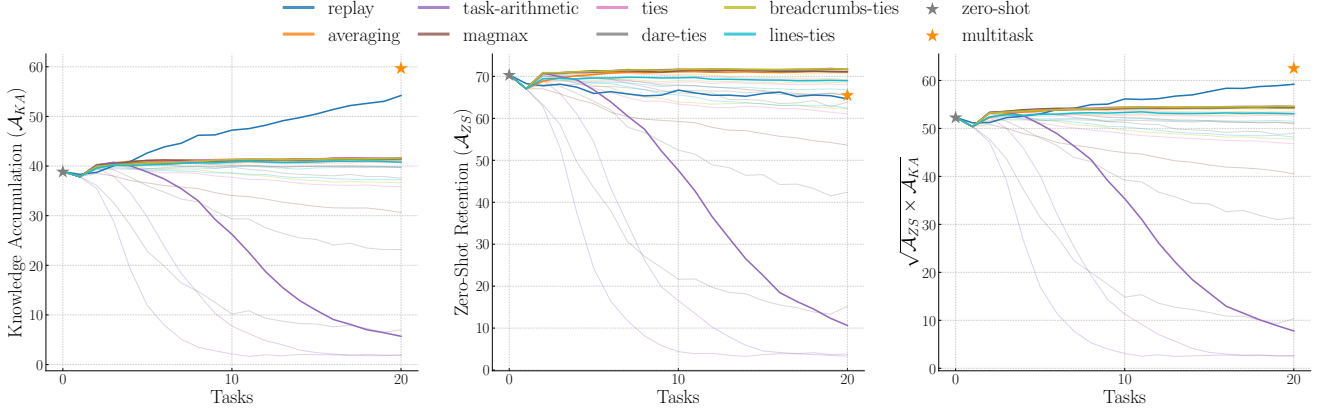


Figure 3. **Offline merging methods struggle with TIME.** All tested merging techniques perform extremely poorly, and are unable to adapt to the temporal setting, underperforming even a simple **replay** baseline that sequentially trains the base model on task-replayed data.

5. **Eval.** The deployed θ_t^O is used for downstream applications and, in our case, extensive evaluation.

Particular choices of init , deploy and f_{merge} correspond to existing approaches, for example $(\text{init}_{\text{ZS}}, \text{deploy}_{\text{ALL}}, f_{\text{merge}}^{\text{WA}})$ simply recovers offline merging through weight averaging over experts models derived from original base weights θ_0 for each task t . Similarly, $(\text{init}_{\text{FT}}, \text{deploy}_{\text{YEMA}}, f_{\text{merge}}^{\text{WA}})$ recovers exponential moving average approaches as done in [68, 76]. Using our framework, we can explore temporal model merging in a systematic manner across various combinations of all three design choices to understand their impact and recommend best practices.

4. Experiments

We first discuss experimental details in Sec. 4.1. Our initial experiments study the applicability of standard, “offline” model merging for the temporal paradigm in Sec. 4.2 and possible extensions in Sec. 4.3. We then transition to a comprehensive exploration of applicable strategies within our TIME framework in Sec. 4.4 and Sec. 4.5, before finally scaling up temporal model merging in Sec. 4.6.

4.1. Experimental Pipeline Details

To study temporal model merging in practically relevant scenarios, we use the continual pretraining benchmark Fomo-in-Flux [68]. It includes 41 adaptation datasets and 22 separate evaluation datasets, covering an array of visual and semantic distribution shifts, alongside well-defined, practically motivated compute constraints for each task.

Training at task t . By default, we continuously fine-tune and merge a ViT-B/16 CLIP [8, 58] model pretrained on LAION-2B [70]. The model is pretrained and fine-tuned using the standard CLIP objective [58]. Following [68], we fix the training steps for each task based on the DataComp-Small computation budget of 1.8×10^9

GFLOPS, split equally across 20 tasks. We also additionally explore longer sequences with $T \in \{50, 100\}$. At each temporal merging step, we allow unrestricted access to a pretraining data pool $\mathcal{P}$, using the same 2M random subset of LAION-400M as in [68]. We use a cosine-decay LR schedule with a linear warmup of 10%, AdamW optimizer [40], a batchsize of 512 and gradient norm clipping to 1. All experiments use PyTorch [53], and are run on a compute cluster using NVIDIA A100/H100s.

Checkpoint Storage. To enable arbitrary merging strategies at any task t , we store each trained expert θ_t^S . We do not store any merged initialization weights θ_t^I or deployed models θ_t^O , as these can be recovered from stored experts.

Evaluation and Metrics. We focus on two key quantities to evaluate temporal model merging: the level of *adaptation*, reflecting performance improvement with each merging step, and *retention*, capturing the preservation of prior knowledge. Specifically, we report two metrics following Roth et al. [68]: Knowledge Accumulation ($\mathcal{A}_{KA}$), the average accuracy (or recall@5 for retrieval) across all 41 adaptation datasets, and Zero-Shot Retention ($\mathcal{A}_{ZS}$), the zero-shot accuracy or recall@5 on all 22 held-out evaluation datasets. Additional details, including a description of our plotting style, can be found in the supplementary.

4.2. Do We Need Model Merging Across Time?

The simplest approach to tackle temporal merging is to disregard the time axis and follow standard *offline* merging paradigms. In TIME terms, this corresponds to a configuration of $(\text{init}_{\text{ZS}}, \text{deploy}_{\text{ALL}}, f_{\text{merge}})$, which always fine-tunes the initial base weights θ_0 . To study the effectiveness of such an offline strategy, we test the above triplet with various choices of f_{merge} , including $f_{\text{merge}}^{\text{WA}}$ (**averaging** in Fig. 3), $f_{\text{merge}}^{\text{TA}}$ (**task-arithmetic**), $f_{\text{merge}}^{\text{Max}}$ (**magmax**), $f_{\text{merge}}^{\text{TIES}}$ (**ties**), $f_{\text{merge}}^{\text{DARE}}$ applied over TIES (**dare-ties**), $f_{\text{merge}}^{\text{Bread}}$ (**breadcrumbs-ties**), and $f_{\text{merge}}^{\text{Lines}}$ (**lines-ties**). To put the results

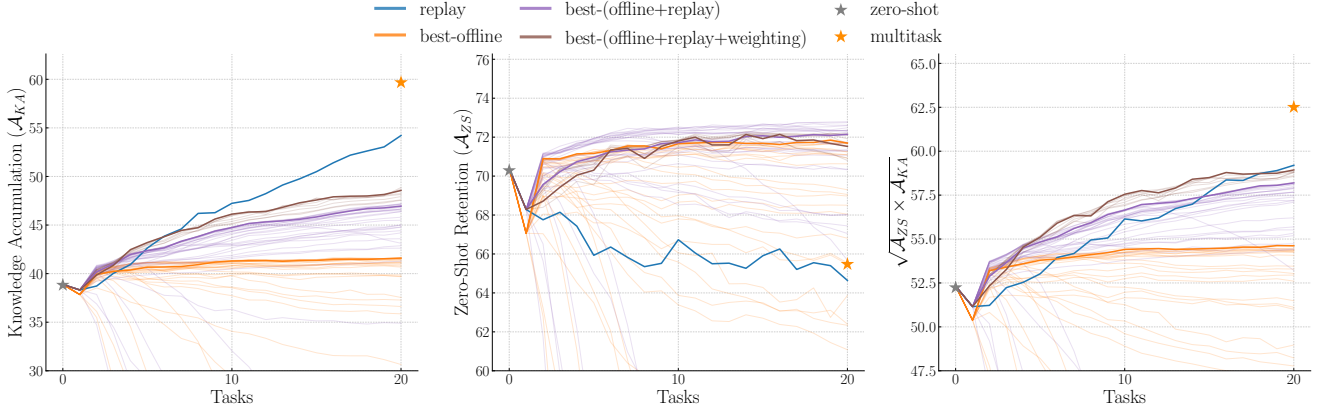


Figure 4. **Improving offline merging.** We identify two simple methods for adapting offline-merging methods to the temporal setting: (1) replaying data from previous tasks (**best-(offline+replay)**) and (2) recency-biased weighting of task checkpoints (**best-(offline+replay+weighting)**). With these method improvements, offline merging methods can match the **replay** baseline.

in context, we incorporate (1) a simple continual fine-tuning baseline (**replay**), following Sec. 4.1, which replays on both pretraining and previous task data, (2) initial zero-shot (θ_0) performance lower bound, and (3) multitask training upper bound. We visualize trajectories over time for knowledge accumulation $\mathcal{A}_{KA}$, zero-shot retention $\mathcal{A}_{ZS}$, and the geometric mean of both in Fig. 3. Our results clearly highlight that there are only marginal differences between merging techniques f_{merge} when deployed in an offline manner for a temporal problem, and they all trace similar trajectories in the $\mathcal{A}_{KA}$ and $\mathcal{A}_{ZS}$ space and achieve similar final performance. We do find that the vanilla $f_{\text{merge}}^{\text{TA}}$ (**task-arithmetic**) starts to deviate after a few update steps, an issue fixed by the various subsequent extensions such as TIES, DARE or LiNeS. Overall, however, unlike straightforward continual fine-tuning (**replay**), offline merging with *any* technique fails to address the temporal aspects of the problem, particularly struggling to consistently acquire new knowledge over time (as observed in Fig. 3, left).

Key takeaway: offline merging with *any* technique yields similar results, failing to add new knowledge over time.

4.3. Can Replay or Time-Weighting Fix the Issue?

We next ask: “What are some simple extensions to offline methods to close the gap to the replay baseline”? As the continual fine-tuning baseline *replays* on past data from all previous tasks while training at the current task t , can this task data-mixing also help offline merging methods?

Data replaying improves offline merging. Since offline methods operate entirely under a task-independent assumption, they fail to capture any temporal dependencies. Fig. 4 shows that simply applying data-replay on top of standard offline merging leads to significant boosts in the overall performance. For instance, **best-(offline+replay)** achieves 58.2% compared to **best-offline** at 54.6%, bringing it closer to the **replay** baseline. However, a notable performance gap

remains, with **best-(offline+replay)** at 58.2% falling short of **replay** at 59.1%.

Recency-biased weighting helps. Next, unlike in standard *offline* averaging, where all task checkpoints are weighted uniformly, we impose temporal ordering via non-uniform weighting for offline merging. We explore several recency-biased, non-uniform weighting schemes, assigning higher weights to more recent tasks to account for the temporal nature of the setting.

We explore various discounting schemes: logarithmic, quadratic, exponential, and cubic, applied to the best offline merge replay method from the previous experiment (please refer to the supplementary for details). As shown in Fig. 4, these schemes improve performance, with **best-(offline+replay+weighting)** reaching 58.9%, yet still falling slightly short of the **replay** baseline at 59.1%. These results provide strong evidence that accounting for the new temporal axis is crucial for effective temporal model merging, even when implemented as an extension of offline merging.

Key Takeaway: Accounting for the time aspect is crucial for effective temporal model merging, even as an extension on top of standard offline merging. Still, a small gap to the simple replay baseline remains.

4.4. TIME Travel

In our effort to adapt offline merging to the temporal setting, we have explored two axes of our TIME framework—merging technique and deployment. As discussed in Sec. 4.2, the choice of merging technique has minimal impact on final performance, while the deployment strategy plays a much more significant role. Next, we systematically explore the design space for temporal merging by testing all valid combinations of three initialization protocols and three deployment protocols described in Sec. 3.2. After discarding incompatible pairs, such as `initZS` with `deployFT`, we evaluated the remaining eight variants us-

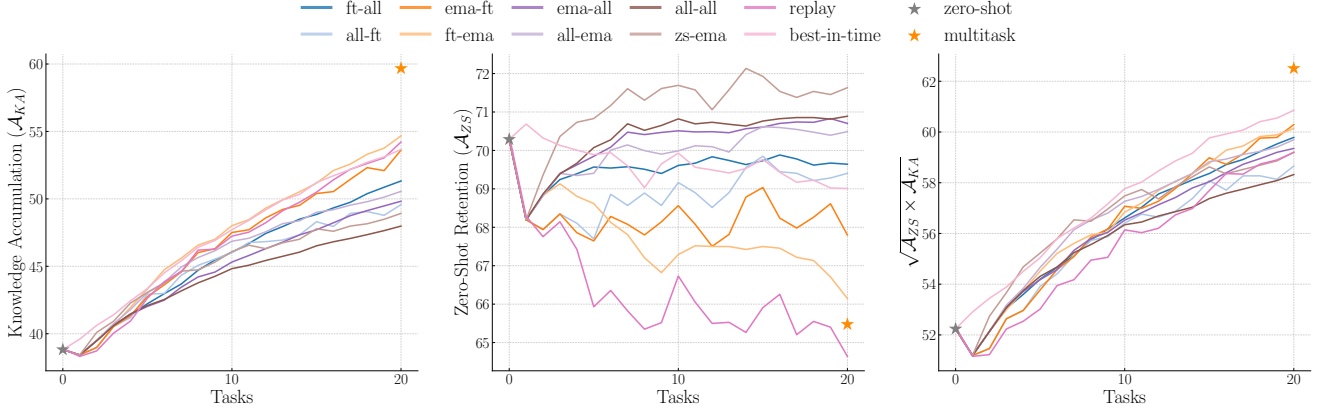


Figure 5. **A journey through TIME.** We explore various initialization and deployment protocols, finding that the EMA initialization-deployment strikes the best balance between knowledge accumulation and zero-shot retention. We refer to this strategy as *Best-in-TIME*.

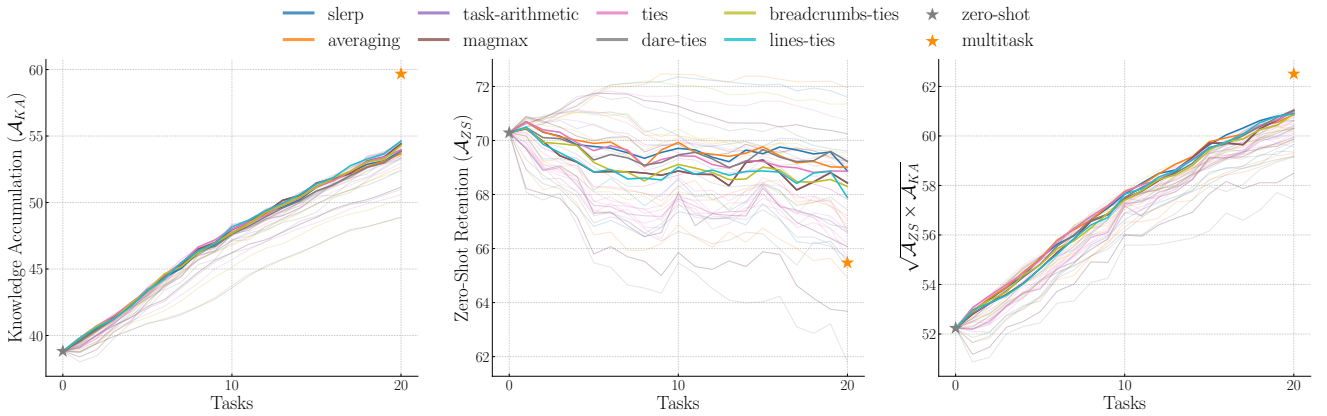


Figure 6. **Sweeping *Best-in-TIME*.** All merging techniques perform well with the *Best-in-TIME* strategy. Indeed, there are no significant differences between techniques, indicating that initialization and deployment matter more for temporal merging.

ing weight averaging as the merging technique. As shown in Fig. 5, the choice of initialization and deployment strategy largely determines performance, significantly affecting both knowledge accumulation and retention. One combination that stands out consistently is init_{EMA} alongside $\text{deploy}_{\text{EMA}}$. This supports the findings of [68, 76] on the applicability of EMA-like solutions to small-scale continual learning and pretraining, while extending the analysis to a much larger, more systematic study of temporal model merging.

Since EMA-style model merging achieves a better balance between knowledge accumulation and retention than other methods, we call this approach *Best-in-TIME*. In the next section, we will explore the robustness of this strategy across different merging techniques.

Key Takeaway: The EMA strategy for initialization and deployment consistently performs the best.

4.5. What is the best merge for Best-in-TIME?

Having identified the optimal initialization and deployment merging strategy in the previous section using weight-averaging as a canonical merging technique, we now inves-

tigate the robustness of our finding by sweeping over other merging techniques, similar to the offline-merging analysis in Sec. 4.2. In particular, we test 7 different merging techniques while keeping the *Best-in-TIME* initialization and deployment strategy. From Fig. 6, it is immediately evident that all merging techniques perform very similarly—in fact, the knowledge accumulation ($\mathcal{A}_{KA}$) of all techniques lie within a single standard deviation of performance variation across multiple runs. This indicates that, over a sufficiently long time horizon, all techniques converge to a similar behavior, echoing our results in Sec. 4.2. However, we do notice higher variance in the retention metric ($\mathcal{A}_{ZS}$). Overall, our results suggest that for temporal model merging, the specific merging technique used is much less important than selecting the best initialization and deployment strategies.

Key Takeaway: The choice of merging technique is not particularly important—all techniques perform similarly.

4.6. Scaling Up Temporal Model Merging

We next study the scaling behaviour of temporal model merging across three-axes: *model size*, *compute budget*, and *number of tasks* (results in Fig. 7 and supplemen-

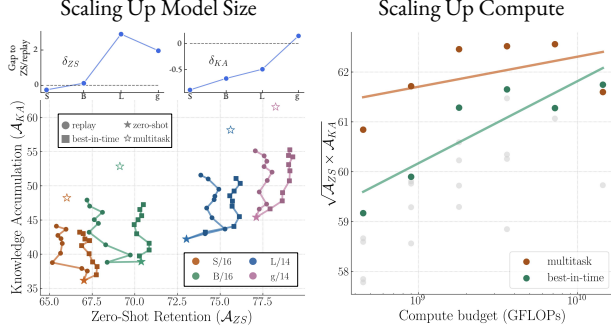


Figure 7. **Scaling up model merging.** (left) With scale, we observe continued improvements of model merging compared to the standard replay baseline. (right) Our *Best-in-TIME* method continues to improve with scaled total compute budget moving close to the multitask upper-bound. Gray points in the plot visualize suboptimal *Best-in-TIME* hyperparameter-instantiations.

tary). All our experiments utilize the *Best-in-TIME* setup described previously, conducting hyperparameter-optimal EMA-weight-averaging at each task.

Scaling the Model. As we increase the model scale from S/16 (62.3M parameters) to B/16 (149.6M), L/14 (427.6M), and finally g/14 (1.37B) in Fig. 7 (left), we compare sequential fine-tuning (circles) and *Best-in-TIME* (squares) in terms of knowledge accumulation and retention. We find that: (1) *Best-in-TIME* scales well with model size, with larger models exhibiting increased affinity to temporal merging. This extends and further corroborates standard offline merging insights by Yadav et al. [89], who showed that model scale facilitates merging. (2) Moreover, while Roth et al. [68] highlight better continual fine-tuning with scale (as also studied by Ibrahim et al. [23]), we show temporal model merging to be substantially more effective across scale. For larger models all the way to the largest ViT-g/14, *Best-in-TIME* vastly outperforms or matches sequential fine-tuning and the multitask target in knowledge retention and positive backward transfer. Moreover, scale facilitates equivalent degrees of knowledge accumulation between sequential fine-tuning and temporal model merging. Therefore, our model scaling results strongly advocate the use of temporal model merging solutions over standard continual fine-tuning methods [23, 68].

Scaling the Compute. Keeping the model fixed to ViT-B/16, we next scale up the available compute budget by increasing the number of allocated update steps per task. We compare a multitask model, trained on all tasks simultaneously, to a budget-optimal *Best-in-TIME*. The only hyperparameter for *Best-in-TIME* is the interpolation weight w . For each compute budget, there is a clear optimal choice of that hyperparameter (suboptimal runs shown as gray dots in Fig. 7 (right)). Higher values of w put greater emphasis on accumulation, allowing optimal accumulation-retention

trade-offs to be reached at lower compute budgets. However, if a larger compute budget is available, less aggressive temporal model merging can achieve higher absolute trade-offs. Note that in Fig. 7 (right), we report the geometric mean between accumulation and retention, corresponding to the right-most panel in previous plots. Our results indicate that *Best-in-TIME* scales very well across compute budgets, *clearly approaching the multitask upper bound* in accumulation-retention balance at larger compute budgets. Put together, this gives practitioners the flexibility to adjust *Best-in-TIME* to their compute budget through the choice of w . Moreover, with more compute to train each respective expert model, the merging process becomes more effective.

Scaling the Number of Tasks. We next study how *Best-in-TIME* performs as we increase the number of merging time-steps from $T=20$ to $T=50$ and $T=100$. As shown in Figs. 10 and 11, *Best-in-TIME* remains the optimal method across different initialization and deployment strategies. For $T=50$ and $T=100$, we notice that other methods are initially competitive, but face a significant drop-off in performance near the end of the task sequence, while *best-in-TIME* continues to improve monotonically. This further corroborates that our *Best-in-TIME* is an extremely scalable method for temporal model merging.

Key Takeaway: Temporal model merging with *Best-in-TIME* scales efficiently across model sizes and the number of tasks. Moreover, compute scaling improves the effectiveness of temporal model merging.

5. Conclusion

In this work, we study *temporal model merging*, addressing the challenge of continually merging multimodal models as new tasks and data arrive, and new expert models are trained in succession. To formalize this setting, we propose TIME, a novel unifying framework breaking down temporal model merging into three key axes: (1) initialization phase defining starting weights before each task, (2) deployment phase denoting post-training expert model aggregation, and (3) the choice of merging technique. Using TIME, we conduct a large-scale systematic study uncovering crucial practical guidelines for temporal model merging. Our experiments on the FoMo-in-Flux benchmark spanning 63 datasets, showcase that accounting for the temporal aspect is crucial, with standard offline merging techniques falling short in this dynamic setting. Moreover, we find the particular choice of merging technique matters far less than the merging strategy for initialization and deployment. Finally, we introduce *Best-in-TIME*, which scales favorably with model size and outperforms existing methods for continual multimodal pretraining. Together, our work provides a systematic entry point into temporal model merging and establishes best practices for this emerging field.

Acknowledgements. The authors would like to thank (in random order) Shyamgopal Karthik, Shashwat Goel, Ankit Sonthalia, Olivier Hénaff, Alexandre Ramé, and Daniel Marczak for helpful feedback. The plotting style in our work is inspired by figures from Beyer et al. [4]. The style of Fig. 2 is inspired by Figure 1 of Karamcheti et al. [30]. VU, KR, and SD thank the International Max Planck Research School for Intelligent Systems (IMPRS-IS). VU, KR, and SD also thank the European Laboratory for Learning and Intelligent Systems (ELLIS) PhD program for support. VU is supported by a Google PhD Fellowship in Machine Intelligence. SA is supported by a Newton Trust Grant. AP is supported by the Federal Ministry of Education and Research (BMBF), FKZ: 011524085B. MB acknowledges financial support via the Open Philanthropy Foundation funded by the Good Ventures Foundation. MB is a member of the Machine Learning Cluster of Excellence, funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany’s Excellence Strategy – EXC number 2064/1 – Project number 390727645. ZA acknowledges the support from the German Research Foundation (DFG): SFB 1233, Robust Vision: Inference Principles and Neural Mechanisms, project number: 276693517 and ERC Grant DEXIM, project number: 853489. This research utilized compute resources at the Tübingen Machine Learning Cloud, DFG FKZ INST 37/1057-1 FUGG.

References

- [1] Samuel Ainsworth, Jonathan Hayase, and Siddhartha Srini-vasa. Git re-basin: Merging models modulo permutation symmetries. In *The Eleventh International Conference on Learning Representations*, 2023. 2
- [2] Takuya Akiba, Makoto Shing, Yujin Tang, Qi Sun, and David Ha. Evolutionary optimization of model merging recipes. *arXiv preprint arXiv:2403.13187*, 2024. 2
- [3] Anton Alexandrov, Veselin Raychev, Mark Niklas Müller, Ce Zhang, Martin Vechev, and Kristina Toutanova. Mitigating catastrophic forgetting in language transfer via model merging. *arXiv preprint arXiv:2407.08699*, 2024. 2
- [4] Lucas Beyer, Xiaohua Zhai, Amélie Royer, Larisa Markeeva, Rohan Anil, and Alexander Kolesnikov. Knowledge distillation: A good teacher is patient and consistent. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10925–10934, 2022. 9, 14
- [5] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ B. Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, Shyamal Buch, Dallas Card, Rodrigo Castellon, Niladri S. Chatterji, Annie S. Chen, Kathleen Creel, Jared Quincy Davis, Dorottya Demszky, Chris Donahue, Moussa Doumbouya, Esin Durmus, Stefano Ermon, John Etchemendy, Kawin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren Gillespie, Karan Goel, Noah D. Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto, Peter Henderson, John Hewitt, Daniel E. Ho, Jenny Hong, Kyle Hsu, Jing Huang, Thomas Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, Omar Khat-tab, Pang Wei Koh, Mark S. Krass, Ranjay Krishna, Rohith Kuditipudi, and et al. On the opportunities and risks of foundation models. *CoRR*, abs/2108.07258, 2021. 1
- [6] Jorg Bornschein, Alexandre Galashov, Ross Hemsley, Amal Rannen-Triki, Yutian Chen, Arslan Chaudhry, Xu Owen He, Arthur Douillard, Massimo Caccia, Qixuan Feng, et al. Nevis’ 22: A stream of 100 tasks sampled from 30 years of computer vision research. *JMLR*, 2023. 2
- [7] Zhipeng Cai, Ozan Sener, and Vladlen Koltun. Online continual learning with natural distribution shifts: An empirical study with visual data. In *ICCV*, 2021. 2
- [8] Mehdi Cherti, Romain Beaumont, Ross Wightman, Mitchell Wortsman, Gabriel Ilharco, Cade Gordon, Christoph Schuhmann, Ludwig Schmidt, and Jenia Jitsev. Reproducible scaling laws for contrastive language-image learning. *arXiv preprint arXiv:2212.07143*, 2022. 5
- [9] Leshem Choshen, Elad Venezian, Shachar Don-Yehia, Noam Slonim, and Yoav Katz. Where to start? analyzing the potential value of intermediate models. *arXiv preprint arXiv:2211.00107*, 2022. 2
- [10] Geoffrey Cideron, Andrea Agostinelli, Johan Ferret, Sertan Girgin, Romuald Elie, Olivier Bachem, Sarah Perrin, and Alexandre Ramé. Diversity-rewarded cfg distillation. *arXiv preprint arXiv:2410.06084*, 2024. 2
- [11] MohammadReza Davari and Eugene Belilovsky. Model breadcrumbs: Scaling multi-task model merging with sparse masks. In *Computer Vision – ECCV 2024*, pages 270–287, Cham, 2025. Springer Nature Switzerland. 2, 4
- [12] Pala Tej Deep, Rishabh Bhardwaj, and Soujanya Poria. Della-merging: Reducing interference in model merging through magnitude-based sampling. *arXiv preprint arXiv:2406.11617*, 2024. 2
- [13] Shachar Don-Yehiya, Elad Venezian, Colin Raffel, Noam Slonim, Yoav Katz, and Leshem Choshen. Cold fusion: Collaborative descent for distributed multitask finetuning. *arXiv preprint arXiv:2212.01378*, 2022. 1, 2
- [14] Jonathan Frankle, Gintare Karolina Dziugaite, Daniel Roy, and Michael Carbin. Linear mode connectivity and the lottery ticket hypothesis. In *Proceedings of the 37th International Conference on Machine Learning*, pages 3259–3269. PMLR, 2020. 2
- [15] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM computing surveys*, 2014. 3
- [16] Saurabh Garg, Mehrdad Farajtabar, Hadi Pouransari, Raviteja Vemulapalli, Sachin Mehta, Oncel Tuzel, Vaishaal Shankar, and Fartash Faghri. Tic-clip: Continual training of clip models. In *ICLR*, 2024. 2
- [17] Timur Garipov, Pavel Izmailov, Dmitrii Podoprikin, Dmitry P Vetrov, and Andrew G Wilson. Loss surfaces, mode connectivity, and fast ensembling of dnns. In *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2018. 1

- [18] Charles Goddard, Shamane Siriwardhana, Malikeh Ehghaghi, Luke Meyers, Vlad Karpukhin, Brian Benedict, Mark McQuade, and Jacob Solawetz. Arcee’s mergekit: A toolkit for merging large language models. *arXiv preprint arXiv:2403.13257*, 2024. 2
- [19] Alexey Gorbатовski, Boris Shaposhnikov, Alexey Malakhov, Nikita Surnachev, Yaroslav Aksenov, Ian Maksimov, Nikita Balagansky, and Daniil Gavrilov. Learn your reference model for real good alignment. *arXiv preprint arXiv:2404.09656*, 2024. 2
- [20] Zhongrui Gui, Shuyang Sun, Runjia Li, Jianhao Yuan, Zhaochong An, Karsten Roth, Ameya Prabhu, and Philip Torr. knn-clip: Retrieval enables training-free segmentation on continually expanding large vocabularies. *arXiv preprint arXiv:2404.09447*, 2024. 1, 2
- [21] Yifei He, Yuzheng Hu, Yong Lin, Tong Zhang, and Han Zhao. Localize-and-stitch: Efficient model merging via sparse task arithmetic. *arXiv preprint arXiv:2408.13656*, 2024. 2
- [22] Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. 1
- [23] Adam Ibrahim, Benjamin Thérien, Kshitij Gupta, Mats L Richter, Quentin Anthony, Timothée Lesort, Eugene Belilovsky, and Irina Rish. Simple and scalable strategies to continually pre-train large language models. *arXiv preprint arXiv:2403.08763*, 2024. 2, 8
- [24] Gabriel Ilharco, Mitchell Wortsman, Samir Yitzhak Gadre, Shuran Song, Hannaneh Hajishirzi, Simon Kornblith, Ali Farhadi, and Ludwig Schmidt. Patching open-vocabulary models by interpolating weights. In *NeurIPS*, 2022. 1, 2, 4, 14
- [25] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *The Eleventh International Conference on Learning Representations*, 2023. 2, 4, 14
- [26] Pavel Izmailov, Dmitrii Podoprikin, Timur Garipov, Dmitry Vetrov, and Andrew Gordon Wilson. Averaging weights leads to wider optima and better generalization. *arXiv preprint arXiv:1803.05407*, 2018. 2
- [27] Dong-Hwan Jang, Sangdoo Yun, and Dongyoon Han. Model stock: All we need is just a few fine-tuned models. In *Computer Vision – ECCV 2024: 18th European Conference, Milan, Italy, September 29–October 4, 2024, Proceedings, Part XLIV*, page 207–223, Berlin, Heidelberg, 2024. Springer-Verlag. 1, 4
- [28] Xisen Jin, Xiang Ren, Daniel Preotiuc-Pietro, and Pengxiang Cheng. Dataless knowledge fusion by merging weights of language models. In *The Eleventh International Conference on Learning Representations*, 2023. 2
- [29] Jean Kaddour. Stop wasting my time! saving days of imagenet and bert training with latest weight averaging. *arXiv preprint arXiv:2209.14981*, 2022. 2
- [30] Siddharth Karamcheti, Suraj Nair, Ashwin Balakrishna, Percy Liang, Thomas Kollar, and Dorsa Sadigh. Prismatic vlms: Investigating the design space of visually-conditioned language models. *arXiv preprint arXiv:2402.07865*, 2024. 9
- [31] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *PNAS*, 2017. 3
- [32] Pang Wei Koh, Shiori Sagawa, Henrik Marklund, Sang Michael Xie, Marvin Zhang, Akshay Balsubramani, Weihua Hu, Michihiro Yasunaga, Richard Lanus Phillips, Irena Gao, et al. Wilds: A benchmark of in-the-wild distribution shifts. In *International conference on machine learning*, pages 5637–5664. PMLR, 2021. 1
- [33] Jeffrey Li, Mohammadreza Armandpour, Seyed Iman Mirzadeh, Sachin Mehta, Vaishaal Shankar, Raviteja Vemulapalli, Oncel Tuzel, Mehrdad Farajtabar, Hadi Pouransari, and Fartash Faghri. Tic-lm: A multi-year benchmark for continual pretraining of language models. In *NeurIPS 2024 Workshop on Scalable Continual Learning for Lifelong Foundation Models*. 2
- [34] Margaret Li, Suchin Gururangan, Tim Dettmers, Mike Lewis, Tim Althoff, Noah A Smith, and Luke Zettlemoyer. Branch-train-merge: Embarrassingly parallel training of expert language models. *arXiv preprint arXiv:2208.03306*, 2022. 1
- [35] Tao Li, Weisen Jiang, Fanghui Liu, Xiaolin Huang, and James T Kwok. Learning scalable model soup on a single gpu: An efficient subspace training strategy. *arXiv preprint arXiv:2407.03641*, 2024. 2
- [36] Yong Lin, Hangyu Lin, Wei Xiong, Shizhe Diao, Jianmeng Liu, Jipeng Zhang, Rui Pan, Haoxiang Wang, Wenbin Hu, Hanning Zhang, Hanze Dong, Renjie Pi, Han Zhao, Nan Jiang, Heng Ji, Yuan Yao, and Tong Zhang. Mitigating the alignment tax of RLHF. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 580–606, Miami, Florida, USA, 2024. Association for Computational Linguistics. 2
- [37] Zhiqiu Lin, Jia Shi, Deepak Pathak, and Deva Ramanan. The clear benchmark: Continual learning on real-world imagery. In *NeurIPS*, 2021. 2
- [38] Adam Liska, Tomas Kocisky, Elena Gribovskaya, Tayfun Terzi, Eren Sezener, Devang Agrawal, D’Autume Cyprien De Masson, Tim Scholtes, Manzil Zaheer, Susannah Young, et al. Streamingqa: A benchmark for adaptation to new knowledge over time in question answering models. In *International Conference on Machine Learning*, pages 13604–13622. PMLR, 2022. 2
- [39] Tianlin Liu, Shangmin Guo, Leonardo Bianco, Daniele Calandriello, Quentin Berthet, Felipe Llinares-López, Jessica Hoffmann, Lucas Dixon, Michal Valko, and Mathieu Blondel. Decoding-time realignment of language models. In *Proceedings of the 41st International Conference on Machine Learning*, pages 31015–31031. PMLR, 2024. 2
- [40] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017. 5
- [41] Jinliang Lu, Ziliang Pang, Min Xiao, Yaochen Zhu, Rui Xia, and Jiajun Zhang. Merge, ensemble, and cooperate!

- a survey on collaborative strategies in the era of large language models. *arXiv preprint arXiv:2407.06089*, 2024. 2
- [42] Daniel Marczak, Bartłomiej Twardowski, Tomasz Trzcinski, and Sebastian Cygert. Magmax: Leveraging model merging for seamless continual learning. *arXiv preprint arXiv:2407.06322*, 2024. 2, 4
- [43] Michael S Matena and Colin Raffel. Merging models with fisher-weighted averaging. In *Advances in Neural Information Processing Systems*, 2022. 2
- [44] Matías Mendieta, Boran Han, Xingjian Shi, Yi Zhu, and Chen Chen. Towards geospatial foundation models via continual pretraining. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 16806–16816, 2023. 2
- [45] Sachit Menon and Carl Vondrick. Visual classification via description from large language models. In *The Eleventh International Conference on Learning Representations*, 2023. 1
- [46] Remi Munos, Michal Valko, Daniele Calandriello, Mohammad Gheshlaghi Azar, Mark Rowland, Zhaohan Daniel Guo, Yunhao Tang, Matthieu Geist, Thomas Mesnard, Côme Fiegel, Andrea Michi, Marco Selvi, Sertan Girgin, Nikola Momchev, Olivier Bachem, Daniel J Mankowitz, Doina Precup, and Bilal Piot. Nash learning from human feedback. In *Proceedings of the 41st International Conference on Machine Learning*, pages 36743–36768. PMLR, 2024. 2
- [47] Anshul Nasery, Jonathan Hayase, Pang Wei Koh, and Se-woong Oh. Pleas—merging models with permutations and least squares. *arXiv preprint arXiv:2407.02447*, 2024. 2
- [48] Behnam Neyshabur, Hanie Sedghi, and Chiyuan Zhang. What is being transferred in transfer learning? In *Advances in Neural Information Processing Systems*, pages 512–523. Curran Associates, Inc., 2020. 2, 4
- [49] Kai Nylund, Suchin Gururangan, and Noah A Smith. Time is encoded in the weights of finetuned language models. *arXiv preprint arXiv:2312.13401*, 2023. 2
- [50] Changdae Oh, Yixuan Li, Kyungwoo Song, Sangdoo Yun, and Dongyoon Han. Dawin: Training-free dynamic weight interpolation for robust adaptation. *arXiv preprint arXiv:2410.03782*, 2024. 2
- [51] Oleksiy Ostapenko, Timothee Lesort, Pau Rodríguez, Md Rifat Arefin, Arthur Douillard, Irina Rish, and Laurent Charlin. Continual learning with foundation models: An empirical study of latent replay. In *Conference on Lifelong Learning Agents (CoLLAs)*, 2022. 2
- [52] Jyothish Pari, Samy Jelassi, and Pulkit Agrawal. Collective model intelligence requires compatible specialization. *arXiv preprint arXiv:2411.02207*, 2024. 2
- [53] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019. 5
- [54] Ameya Prabhu, Zhipeng Cai, Puneet Dokania, Philip Torr, Vladlen Koltun, and Ozan Sener. Online continual learning without the storage constraint. *arXiv preprint arXiv:2305.09253*, 2023. 2, 3
- [55] Ameya Prabhu, Hasan Abed Al Kader Hammoud, Puneet Dokania, Philip HS Torr, Ser-Nam Lim, Bernard Ghanem, and Adel Bibi. Computationally budgeted continual learning: What does matter? In *CVPR*, 2023. 2, 3
- [56] Ameya Prabhu, Hasan Abed Al Kader Hammoud, Ser-Nam Lim, Bernard Ghanem, Philip HS Torr, and Adel Bibi. From categories to classifier: Name-only continual learning by exploring the web. *arXiv preprint arXiv:2311.11293*, 2023. 2
- [57] Sarah Pratt, Ian Covert, Rosanne Liu, and Ali Farhadi. What does a platypus look like? generating customized prompts for zero-shot image classification. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 15691–15701, 2023. 1
- [58] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021. 5
- [59] Alexandre Rame, Kartik Ahuja, Jianyu Zhang, Matthieu Cord, Leon Bottou, and David Lopez-Paz. Model rata-touille: Recycling diverse models for out-of-distribution generalization. In *Proceedings of the 40th International Conference on Machine Learning*, pages 28656–28679. PMLR, 2023. 2
- [60] Alexandre Ramé, Johan Ferret, Nino Vieillard, Robert Dadashi, Léonard Hussenot, Pierre-Louis Cedo, Pier Giuseppe Sessa, Sertan Girgin, Arthur Douillard, and Olivier Bachem. Warp: On the benefits of weight averaged rewarded policies. *arXiv preprint arXiv:2406.16768*, 2024. 2, 4
- [61] Alexandre Ramé, Matthieu Kirchmeyer, Thibaud Rahier, Alain Rakotomamonjy, Patrick Gallinari, and Matthieu Cord. Diverse weight averaging for out-of-distribution generalization. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, 2024. 2
- [62] Alexandre Ramé, Nino Vieillard, Léonard Hussenot, Robert Dadashi, Geoffrey Cideron, Olivier Bachem, and Johan Ferret. Warm: On the benefits of weight averaged reward models. *arXiv preprint arXiv:2401.12187*, 2024. 2
- [63] Regent’s, ParkLondon, Ukj, and . Utans. Weight averaging for neural networks and local resampling. 1996. 1, 2, 4
- [64] Mark Rofin, Nikita Balagansky, and Daniil Gavrilov. Linear interpolation in parameter space is good enough for finetuned language models. *arXiv preprint arXiv:2211.12092*, 2022. 1
- [65] Karsten Roth, Jae Myung Kim, A. Sophia Koepke, Oriol Vinyals, Cordelia Schmid, and Zeynep Akata. Waffling around for performance: Visual classification with random words and broad concepts. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 15746–15757, 2023. 1
- [66] Karsten Roth, Zeynep Akata, Dima Damen, Ivana

- Balažević, and Olivier J Hénaff. Context-aware multimodal pretraining. *arXiv preprint arXiv:2411.15099*, 2024. 1
- [67] Karsten Roth, Lukas Thede, A. Sophia Koepke, Oriol Vinyals, Olivier J Henaff, and Zeynep Akata. Fantastic gains and where to find them: On the existence and prospect of general knowledge transfer between any pre-trained model. In *The Twelfth International Conference on Learning Representations*, 2024. 2
- [68] Karsten Roth, Vishaal Udandarao, Sebastian Dziadzio, Ameya Prabhu, Mehdi Cherti, Oriol Vinyals, Olivier Hénaff, Samuel Albanie, Matthias Bethge, and Zeynep Akata. A practitioner’s guide to continual multimodal pre-training. *arXiv preprint arXiv:2408.14471*, 2024. 1, 2, 3, 4, 5, 7, 8, 14
- [69] Sunny Sanyal, Atula Tejaswi Neerkaje, Jean Kaddour, Abhishek Kumar, and sujay sanghavi. Early weight averaging meets high learning rates for LLM pre-training. In *First Conference on Language Modeling*, 2024. 1, 2
- [70] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade Gordon, Ross Wightman, Mehdi Cherti, Theo Coombes, Aarush Katta, Clayton Mullis, Mitchell Wortsman, et al. Laion-5b: An open large-scale dataset for training next generation image-text models. *Advances in Neural Information Processing Systems*, 35:25278–25294, 2022. 5
- [71] Ekansh Sharma, Daniel M Roy, and Gintare Karolina Dziugaite. The non-local model merging problem: Permutation symmetries and variance collapse. *arXiv preprint arXiv:2410.12766*, 2024. 2
- [72] Li Shen, Anke Tang, Enneng Yang, Guibing Guo, Yong Luo, Lefei Zhang, Xiaochun Cao, Bo Du, and Dacheng Tao. Efficient and effective weight-ensembling mixture of experts for multi-task model merging. *arXiv preprint arXiv:2410.21804*, 2024. 2
- [73] Ken Shoemake. Animating rotation with quaternion curves. *Proceedings of the 12th annual conference on Computer graphics and interactive techniques*, 1985. 4
- [74] Shikhar Srivastava, Md Yousuf Harun, Robik Shrestha, and Christopher Kanan. Improving multimodal large language models using continual learning. *arXiv preprint arXiv:2410.19925*, 2024. 2
- [75] George Stoica, Pratik Ramesh, Boglarka Ecsedi, Leshem Choshen, and Judy Hoffman. Model merging with svd to tie the knots. *arXiv preprint arXiv:2410.19735*, 2024. 2
- [76] Zafir Stojanovski, Karsten Roth, and Zeynep Akata. Momentum-based weight interpolation of strong zero-shot models for continual learning. *arXiv preprint arXiv:2211.03186*, 2022. 2, 3, 4, 5, 7
- [77] Yi-Lin Sung, Linjie Li, Kevin Lin, Zhe Gan, Mohit Bansal, and Lijuan Wang. An empirical study of multimodal model merging. *arXiv preprint arXiv:2304.14933*, 2023. 2
- [78] Derek Tam, Mohit Bansal, and Colin Raffel. Merging by matching models in task parameter subspaces. *Transactions on Machine Learning Research*, 2024. 2
- [79] Derek Tam, Yash Kant, Brian Lester, Igor Gilitschenski, and Colin Raffel. Realistic evaluation of model merging for compositional generalization. *arXiv preprint arXiv:2409.18314*, 2024. 2
- [80] Lukas Thede, Karsten Roth, Olivier J Hénaff, Matthias Bethge, and Zeynep Akata. Reflecting on the state of rehearsal-free continual learning with pretrained models. *arXiv preprint arXiv:2406.09384*, 2024. 2
- [81] Vishaal Udandarao, Ankush Gupta, and Samuel Albanie. Sus-x: Training-free name-only transfer of vision-language models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 2725–2736, 2023. 1
- [82] Vishaal Udandarao, Nikhil Parthasarathy, Muhammad Ferjad Naeem, Talfan Evans, Samuel Albanie, Federico Tombari, Yongqin Xian, Alessio Tonioni, and Olivier J Hénaff. Active data curation effectively distills large-scale multimodal models. *arXiv preprint arXiv:2411.18674*, 2024.
- [83] Vishaal Udandarao, Ameya Prabhu, Adhiraj Ghosh, Yash Sharma, Philip HS Torr, Adel Bibi, Samuel Albanie, and Matthias Bethge. No “zero-shot” without exponential data: Pretraining concept frequency determines multimodal model performance. *arXiv preprint arXiv:2404.04125*, 2024. 1
- [84] Ke Wang, Nikolaos Dimitriadis, Alessandro Favero, Guillermo Ortiz-Jimenez, Francois Fleuret, and Pascal Frossard. Lines: Post-training layer scaling prevents forgetting and enhances model merging. *arXiv preprint arXiv:2410.17146*, 2024. 4, 18
- [85] Ke Wang, Nikolaos Dimitriadis, Guillermo Ortiz-Jimenez, François Fleuret, and Pascal Frossard. Localizing task information for improved model merging and compression. *arXiv preprint arXiv:2405.07813*, 2024. 2
- [86] Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *Proceedings of the 39th International Conference on Machine Learning*, pages 23965–23998. PMLR, 2022. 1, 4
- [87] Mitchell Wortsman, Gabriel Ilharco, Jong Wook Kim, Mike Li, Simon Kornblith, Rebecca Roelofs, Raphael Gontijo Lopes, Hannaneh Hajishirzi, Ali Farhadi, Hongseok Namkoong, and Ludwig Schmidt. Robust fine-tuning of zero-shot models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7959–7971, 2022. 1, 3, 4
- [88] Feng Xiong, Runxi Cheng, Wang Chen, Zhanqiu Zhang, Yiwen Guo, Chun Yuan, and Ruifeng Xu. Multi-task model merging via adaptive weight disentanglement. *arXiv preprint arXiv:2411.18729*, 2024. 2
- [89] Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-merging: Resolving interference when merging models. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. 1, 2, 3, 4, 8, 14
- [90] Prateek Yadav, Colin Raffel, Mohammed Muqeeth, Lucas Caccia, Haokun Liu, Tianlong Chen, Mohit Bansal, Leshem Choshen, and Alessandro Sordoni. A survey on

- model moerging: Recycling and routing among specialized experts for collaborative learning. *arXiv preprint arXiv:2408.07057*, 2024. [2](#)
- [91] Prateek Yadav, Tu Vu, Jonathan Lai, Alexandra Chronopoulou, Manaal Faruqui, Mohit Bansal, and Tsendsuren Munkhdalai. What matters for model merging at scale? *arXiv preprint arXiv:2410.03617*, 2024. [2](#), [3](#)
- [92] Enneng Yang, Li Shen, Guibing Guo, Xingwei Wang, Xiaochun Cao, Jie Zhang, and Dacheng Tao. Model merging in llms, mllms, and beyond: Methods, theories, applications and opportunities. *arXiv preprint arXiv:2408.07666*, 2024. [2](#)
- [93] Enneng Yang, Li Shen, Zhenyi Wang, Guibing Guo, Xiaojun Chen, Xingwei Wang, and Dacheng Tao. Representation surgery for multi-task model merging. *arXiv preprint arXiv:2402.02705*, 2024. [2](#)
- [94] Çağatay Yıldız, Nishaanth Kanna Ravichandran, Prishruit Punia, Matthias Bethge, and Beyza Ermis. Investigating continual pretraining in large language models: Insights and implications. *arXiv preprint arXiv:2402.17400*, 2024. [2](#)
- [95] Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *Proceedings of the 41st International Conference on Machine Learning*, pages 57755–57775. PMLR, 2024. [2](#), [4](#)
- [96] Kerem Zaman, Leshem Choshen, and Shashank Srivastava. Fuse to forget: Bias reduction and selective memorization through model fusion. *arXiv preprint arXiv:2311.07682*, 2023. [2](#)
- [97] Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. In *ICML*, 2017. [3](#)
- [98] Renrui Zhang, Rongyao Fang, Peng Gao, Wei Zhang, Kunchang Li, Jifeng Dai, Yu Qiao, and Hongsheng Li. Tip-adapter: Training-free clip-adapter for better vision-language modeling. *arXiv preprint arXiv:2111.03930*, 2021. [1](#)
- [99] Shenghe Zheng and Hongzhi Wang. Free-merging: Fourier transform for model merging with lightweight experts. *arXiv preprint arXiv:2411.16815*, 2024. [2](#)
- [100] Luca Zhou, Daniele Solombrino, Donato Crisostomi, Maria Sofia Bucarelli, Fabrizio Silvestri, and Emanuele Rodolà. Atm: Improving model merging by alternating tuning and merging. *arXiv preprint arXiv:2411.03055*, 2024. [1](#), [2](#)